```
"""Football match model: time-weighted Poisson regression (Dixon-Coles style, no

Each match becomes two rows: (attacking team, defending team, home flag) -> goals
Recent matches count more. Ridge regularisation stops small samples from producin
"""
import numpy as np
import pandas as pd
from sklearn.linear_model import PoissonRegressor

MAX_GOALS = 10
HALF_LIFE_DAYS = 300


def _rows(df: pd.DataFrame) -> pd.DataFrame:
    home = pd.DataFrame({"att": df["Home"], "dfn": df["Away"], "is_home": 1,
                         "goals": df["HG"], "date": df["Date"]})
    away = pd.DataFrame({"att": df["Away"], "dfn": df["Home"], "is_home": 0,
                         "goals": df["AG"], "date": df["Date"]})
    return pd.concat([home, away], ignore_index=True)


class MatchModel:
    def __init__(self, df: pd.DataFrame, half_life: float = HALF_LIFE_DAYS, alpha
        df = df.dropna(subset=["Date", "Home", "Away", "HG", "AG"]).copy()
        if len(df) < 30:
            raise ValueError("Not enough matches to fit a model (need at least 30
        self.teams = sorted(set(df["Home"]) | set(df["Away"]))
        self.last_date = df["Date"].max()

        rows = _rows(df)
        age = (self.last_date - rows["date"]).dt.days.clip(lower=0)
        weights = 0.5 ** (age / half_life)

        X = self._design(rows["att"], rows["dfn"], rows["is_home"])
        self.reg = PoissonRegressor(alpha=alpha / len(rows), max_iter=500)
        self.reg.fit(X, rows["goals"], sample_weight=weights)
        self.n_matches = len(df)

    def _design(self, att, dfn, is_home) -> pd.DataFrame:
        cols = {f"att_{t}": (np.asarray(att) == t).astype(float) for t in self.te
        cols.update({f"def_{t}": (np.asarray(dfn) == t).astype(float) for t in se
        cols["is_home"] = np.asarray(is_home, dtype=float)
        return pd.DataFrame(cols)
```

```
def expected_goals(self, home: str, away: str) -> tuple[float, float]:
    X = self._design([home, away], [away, home], [1, 0])
    lam = self.reg.predict(X)
    return float(lam[0]), float(lam[1])

def predict(self, home: str, away: str) -> dict:
    lh, la = self.expected_goals(home, away)
    k = np.arange(MAX_GOALS + 1)
    fact = np.cumprod(np.maximum(k, 1)).astype(float)  # 0!, 1!, 2!, ...
    ph = np.exp(-lh) * lh ** k / fact
    pa = np.exp(-la) * la ** k / fact
    grid = np.outer(ph, pa)
    grid /= grid.sum()

    home_win = float(np.tril(grid, -1).sum())
    draw = float(np.trace(grid))
    away_win = float(np.triu(grid, 1).sum())
    total = np.add.outer(k, k)
    top = np.dstack(np.unravel_index(np.argsort(grid.ravel())[::-1][:5], grid
    return {
        "xg_home": lh, "xg_away": la,
        "home": home_win, "draw": draw, "away": away_win,
        "over_2_5": float(grid[total > 2.5].sum()),
        "btts": float(grid[1:, 1:].sum()),
        "scorelines": [(int(i), int(j), float(grid[i, j])) for i, j in top],
    }
```